TrustVault

Secure document sharing where the wrong person can never read a file, and you can prove who did.

TrustVault is a document platform for confidential files such as salary letters, contracts and audit papers. Every file is reachable only by the people its owner names. Every attempt to reach it, including the refused ones, is written to a tamper-evident audit trail.

Live application	http://127.0.0.1:5000/
Source code	https://github.com/23jk1a4402-source/TrustVault
Stack	Python 3, Flask, SQLite, server-rendered HTML
Related documents	THREATS.md · PROMPTS.md

1. Demo accounts

One account for each role is available on the live application.

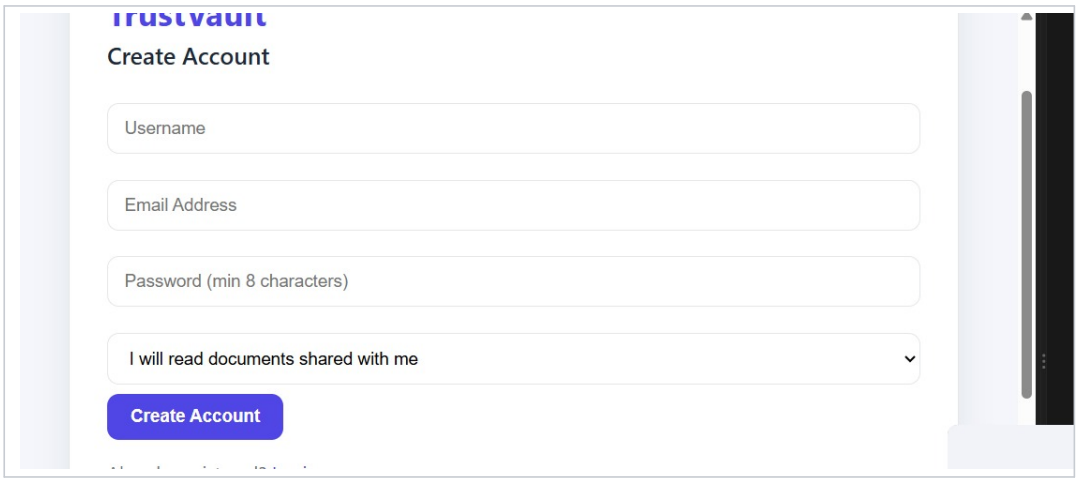
Role	Email	Password
Administrator	admin@gmail.com	password 123
Document owner	23jk1a4402@gmail.com	Gayathri@4402
Viewer	jim@gmail.com	jim123456

These are demonstration accounts that hold no real data.

2. Screenshots

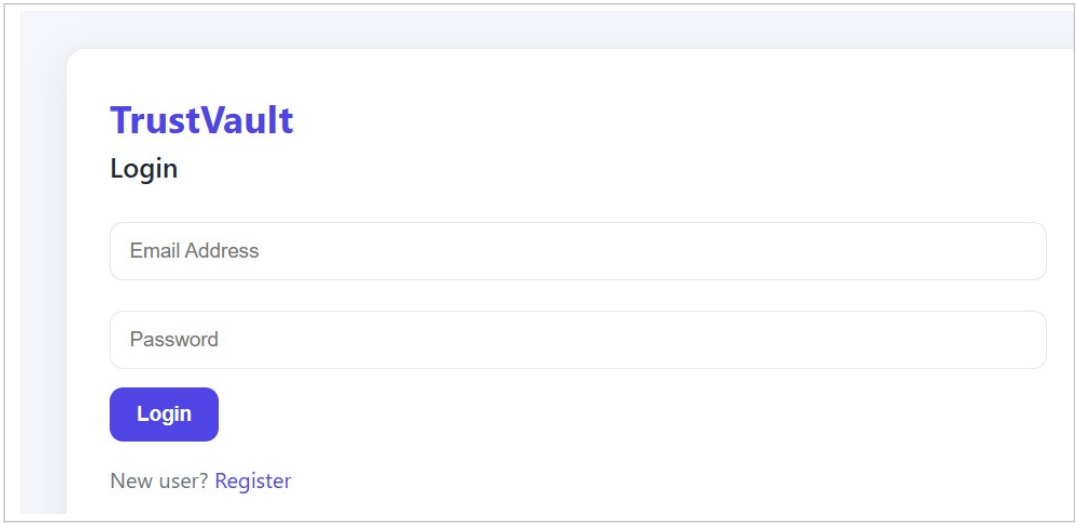
Create an account. Anyone can register as an owner or a viewer. The administrator role cannot be chosen here. The dropdown selects the role.

Registration page and role choice



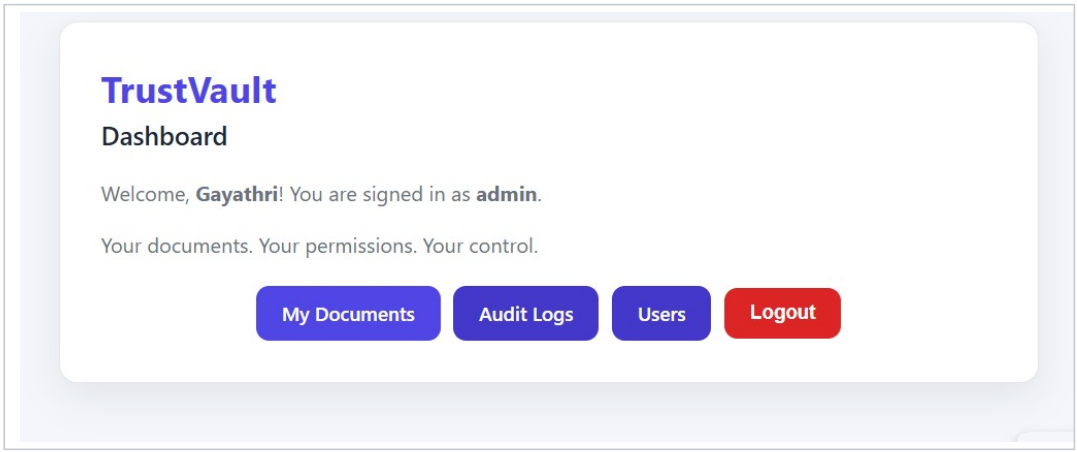
Log in.

Login page



Administrator dashboard. Only an administrator sees the Audit Logs and Users buttons.

Administrator dashboard



3. Roles and what each can do

Capability	Administrator	Owner	Viewer
Register and log in	yes	yes	yes
Upload documents	no	yes	no
Read and download own documents	n/a	yes	n/a
Edit or delete own documents	n/a	yes	n/a
Share a document and take access back	no	yes (own documents only)	no
Read documents shared with them	only if granted	if granted	if granted
Edit documents shared with them	only if granted	if granted with edit	if granted with edit
View the audit log and verify its integrity	yes	no	no
View the user list	yes	no	no
Read the contents of other people's documents	no	no	no

Two design decisions are deliberate:

- **The administrator cannot read document contents.** An administrator runs the platform and reviews activity, but has no more access to a confidential salary letter than any stranger. If an administrator needs a file, the owner must grant it, and that grant is recorded.
- **Nobody can become an administrator through the website.** The role is assigned only from the command line by whoever operates the server (`python app.py make-admin <email>`), and that action is itself written to the audit log.

4. Permission model

Permissions are per-document grants. Each row in the permissions table says: this person may read, or read and edit, this document, with a revoked flag.

```
Request -> logged in? -> role allowed? -> can_access(document, user, action) -> serve
               |              |                          |
               no             no                         no
               v              v                          v
          redirect to     403 + audit               404 + audit
             login           entry                     entry
```

How access is decided:

- **1. One function decides.** can_access() is the single place that answers "may this person do this to this document?". Every route that touches a document calls it. The answer is owner, or has an active grant of the needed level, otherwise no. The default is deny.
- **2. The server decides, every time.** Nothing about access is stored in the session or a token. Each request reads the user, role and grant from the database. Hiding a button is never the protection.
- **3. A refusal looks like "not found".** A person without access gets a 404, exactly as for a document that does not exist, so the existence of a file is not revealed. The refused attempt is still recorded.
- **4. Downloads have a second check.** To download, the person re-enters their account password, and the download route checks access again before sending the file.

What "taking access away immediately" means here

Revoking sets revoked = 1 on the grant. Because every request looks the grant up in the database, the very next request from that person is refused. This works even if they are logged in, have the page open, or hold a download link. The cost is one extra database lookup per request, which is acceptable for SQLite at this scale. A token-based design would be faster but could not revoke immediately.

5. Audit trail

Every significant event is written to the audit_logs table, including refused attempts.

Recorded	Examples
Accounts	successful login, failed login, administrator role granted
Documents	upload, edit, delete, download (after password verification)
Permissions	grant (with level and recipient), revoke
Refusals	DENIED_DOWNLOAD_DOC_7, DENIED_EDIT_DOC_7, DENIED_SHARE_DOC_7, DENIED_ADMIN_PAGE_/audit-logs, DENIED_UPLOAD_NOT_OWNER_ROLE, DENIED_UPLOAD_INVALID_FILE

What stops the log being altered quietly

- **Append-only by the database.** Database triggers refuse every UPDATE and DELETE on audit_logs. Application code has no route that edits it.
- **Hash chain.** Each entry stores the SHA-256 hash of its own content combined with the hash of the entry before it. Changing or removing any earlier entry breaks every hash after it.

- **Verification.** An administrator can press "Check that the log has not been altered". The server recomputes the whole chain and reports either the number of entries verified and the latest fingerprint, or the exact entry where the chain breaks.
- **Writes cannot fork the chain.** Each entry is written inside an immediate transaction, so two simultaneous requests cannot both extend the same hash.

Honest limits. Someone with direct write access to the database file could drop the triggers and rewrite the log. The hash chain exposes a partial edit, but a person who recomputes every hash after the change, or deletes the newest entries, would not be caught by the chain alone. The remedy is to record the latest fingerprint somewhere outside the server. The entry for a download is written when the password check succeeds, immediately before the file is sent, not after the transfer completes.

What the administrator sees

The Audit Logs page lists the latest 300 events, with filters for username, action text and "only refused or failed". It raises an alert for any person with three or more refused or failed events in the last 24 hours. Permission changes appear as GRANT_* and REVOKE_* entries.

6. How the ten requirements are met

#	Requirement	Status	Where
1	Authentication with at least three roles	Done	Administrator, owner, viewer. Passwords are salted hashes.
2	Upload, organise, search and manage documents	Done	Upload, five categories, edit, replace file, delete
3	Per-document read and edit permissions for named people	Done	permissions table, granted by email
4	Owners can grant access and take it back	Done	Share page, Remove access button
5	People without a grant cannot read, download, change or delete	Done	can_access() on every document route
6	Audit trail of uploads, downloads, changes, deletions and permission changes	Done	audit_logs, hash-chained and append-only
7	Search by name, owner, category and date	Done	My Documents search form
8	Validation of what is uploaded	Done	See section 7
9	Administrative view of activity and permission changes	Done	Audit Logs and Users pages
10	Automated tests showing one user cannot reach another's documents	Done	tests/test_security.py, 15 tests

7. File handling

- **Stored outside the web root.** Files live in private_uploads/. They are never served as static files, only through the authenticated download route.
- **Random storage names.** Each file is saved under a random UUID name. The original name is kept only in the database, so a stored path cannot be guessed or used for path tricks.
- **Validation on upload and on replace.** Allowed types: PDF, TXT, PNG, JPG, JPEG, DOC, DOCX, up to 5 MB. The server checks the extension and the first bytes of the file against the type it claims to be, so a program renamed to report.pdf is refused. Empty files are refused. File names are sanitised.
- **Search is safe.** Every filter is passed to the database as a parameter, and % and _ typed by the user are escaped. Search results are restricted to documents the person owns or has been granted.

8. Database

Table	Purpose
users	username, email, password hash, role (admin, owner, viewer)
documents	display name, random storage name, owner, category, upload time
permissions	document, person, level (read or edit), revoked flag. One row per person per document.
audit_logs	user, action, timestamp, previous hash, hash. Protected by append-only triggers.

Tables are created automatically on start, and an older database is upgraded in place without losing data.

9. Running it

Locally

```
python -m venv venv
venv\Scripts\activate     # Windows
pip install -r requirements.txt
python app.py                # http://127.0.0.1:5000
```

Creating an administrator. Register the account on the website, then run:

```
python app.py make-admin admin@gmail.com
```

Automated tests (they use a temporary database and never touch real data):

```
python -m pytest -q
```

Configuration (environment variables)

Variable	Purpose
FLASK_SECRET_KEY	Signs session cookies. Must be set to a long random value in production.
COOKIE_SECURE=1	Sends the session cookie only over HTTPS. Set this when deployed.
FLASK_DEBUG=1	Developer mode. Off by default, and must never be on in production.
TRUSTVAULT_DB, TRUSTVAULT_UPLOADS	Override the database and upload locations

Deployment (PythonAnywhere)

Live at http://127.0.0.1:5000/

- 1. Clone the repository in a Bash console.
- 2. Create a web app with Manual configuration.
- 3. In the WSGI configuration file, set the two environment variables and import the app:

```
import sys, os
sys.path.insert(0, "/home/app1/TrustVault")
os.environ["FLASK_SECRET_KEY"] = "<long random value>"
os.environ["COOKIE_SECURE"] = "1"
from app import app as application
```

- 4. Press Reload. To update, run git pull and reload.

10. API reference: who may call what

A person who is not logged in is redirected to /login by every route marked logged in. A role failure returns 403. A missing document, or one the person has no access to, returns 404.

Method and route	Who may call it
GET /	Anyone

Method and route	Who may call it
GET, POST /register	Anyone. The administrator role cannot be requested.
GET, POST /login	Anyone
POST /logout	Anyone
GET /documents (filters q, owner, category, from, to)	Logged in. Returns only own and granted documents.
POST /documents (upload)	Owner
GET, POST /verify-download/<id>	Document owner, or a person with any active grant
GET /download/<id>	Document owner, or a person with any active grant, after password verification
GET, POST /edit/<id>	Document owner, or a person granted edit
POST /delete/<id>	Document owner only
GET, POST /share/<id>	Document owner only
POST /revoke/<id>/<user_id>	Document owner only
GET /audit-logs	Administrator
POST /audit-logs/verify	Administrator
GET /admin/users	Administrator

11. What was deliberately not built

A smaller system whose permission model holds is better than a larger one with a hole in it. The following were left out on purpose:

- **Folders, groups and inheritance.** These make "who can see this?" harder to answer. Per-document grants are simple to reason about and to test.
- **Public or external share links.** A link that works without an account is hard to revoke and hard to attribute in the audit log.
- **File versions and recovery.** Delete is permanent for the file. The audit log keeps the record that it happened.
- **Encryption of files at rest and malware scanning.**
- **AI features.** None are included in the product.

12. Known gaps and unfinished work

Stated plainly so they are not discovered by surprise. Details and reasoning are in THREATS.md.

- **No CSRF tokens.** Protection relies on the session cookie being SameSite=Lax, which stops browsers sending it on cross-site form posts.
- **No login rate limiting or lockout.** Failed logins are recorded and surfaced to the administrator, but not blocked.
- **Account and email enumeration.** Registration and the share form reveal whether a username or email already exists.
- **No security response headers** such as a Content Security Policy.
- **The administrator cannot deactivate users or change roles** from the interface. Role changes need command-line access.
- **No password reset or email verification.**
- **Audit log tail truncation** is not detectable without a fingerprint stored outside the server (see section 5).
- **SQLite allows one writer at a time,** which suits this scale but not heavy concurrent use.

13. Project layout

Path	Contents
app.py	application, routes and database setup
static/style.css	styles
tests/test_security.py	automated security tests
requirements.txt	dependencies
docs/screenshots/	images used in this document
THREATS.md	threat model and what was done about each threat
PROMPTS.md	how AI was used, including where it was wrong